

CSE/DSC 234 Project 2 Report

a. Training Data Methodology

Our starting point was the released training split of 301 examples. We did not treat that split as a fixed prompt-only benchmark; instead, we made deliberate decisions about schema serialization, target formatting, and training-set variants.

The first major choice was schema serialization. We implemented two dataset variants in `scripts/prepare_data.py` and `src/schema_linking/data_prep.py`:

- `filtered`: serialize only a question-conditioned subset of the schema
- `full_schema`: serialize the full database schema

For the filtered variant, we first ran a deterministic schema filter and then added back any gold tables or columns that the filter missed during training. This mattered because it let us train on shorter prompts without ever hiding the correct supervision target from the model. The actual filter is in `src/schema_linking/filtering.py`. It tokenizes the question, scores table and column names by exact and partial lexical overlap, keeps the top tables, and widens only when the question has weak lexical evidence.

The second major choice was output format. We trained the model to emit compact JSON only, with tables and columns sorted deterministically. The canonical target serialization is implemented in `src/schema_linking/decoding.py`. This made decoding more stable and reduced superficial variation across equivalent outputs.

Our base supervised dataset therefore had:

- Original training size: 301 examples
- Filtered-schema JSONL: `artifacts/prepared/train_filtered.jsonl`
- Full-schema JSONL: `artifacts/prepared/train_full_schema.jsonl`

We also implemented optional augmentation and balancing utilities later in the project to test whether data-centric modifications could help. These utilities live in `scripts/build_training_data_variants.py` and `src/schema_linking/augmentation.py`. The important methodological point is that these variants were fully scripted, deterministic, and auditable rather than hand-wavy data inflation. They produce:

- `augmented_train.json`: deterministic paraphrases of every released training question
- `balanced_train.json`: oversampling for underrepresented databases plus extra weight for multi-table examples
- `augmented_balanced_train.json`: the combination of both

The paraphrase step was intentionally conservative. It applies a small fixed set of lexical rewrites such as `show` $\rightarrow$ `list`, `find` $\rightarrow$ `identify`, and `how many` $\rightarrow$ `count`, and otherwise falls back to a templated rewrite. This gave us a documented augmentation with predictable semantics instead of large-model paraphrases that would have been harder to justify or reproduce.

The balancing step was also not blind duplication: every oversampled copy retained provenance metadata such as `source_question_id`, `sampling_type`, and `sampling_repeat_index`, and the duplication factor increased only for underrepresented databases and structurally harder supervision targets such as multi-table or multi-column schema links.

The resulting sizes were:

- Original: 301
- Augmented: 602
- Balanced: 410
- Augmented + balanced: 724

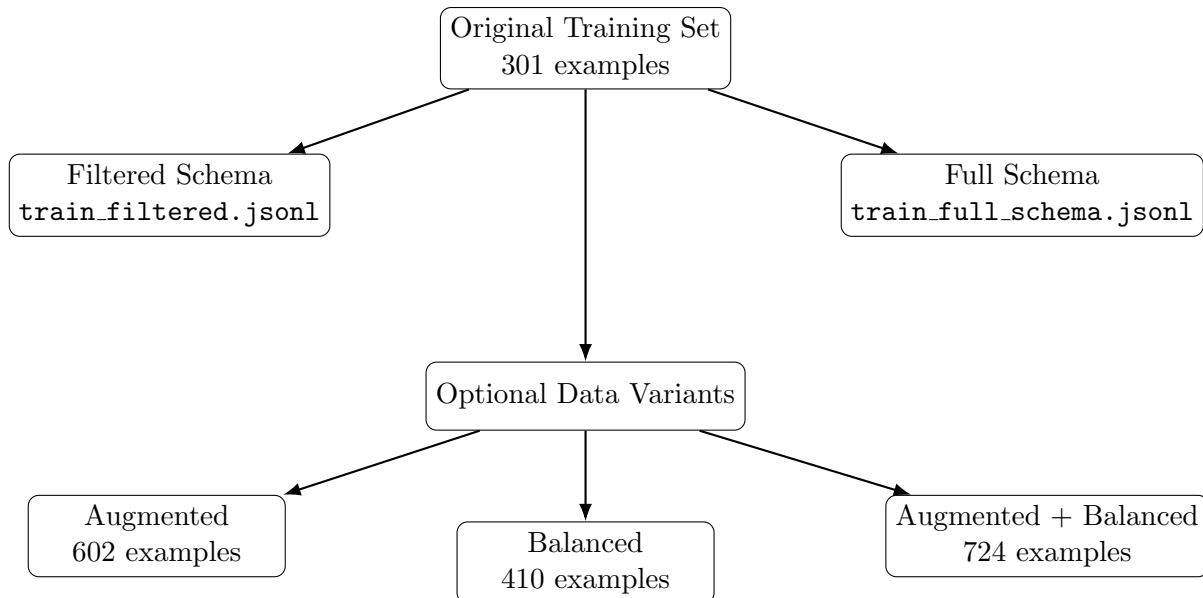


Figure 1: Training data preparation variants. The final model used the filtered-schema version of the original 301-example training set; augmented and balanced variants were kept as ablations because they reduced validation performance.

However, these additional datasets were not used in the final model because they hurt validation performance. In particular, the augmented-balanced variant increased recall but sharply reduced precision, which is costly under the leaderboard metric. This is exactly why I kept augmentation as an explicit ablation instead of folding it silently into the final train set: the report should show that more data was not automatically better, and that the final choice was made from measured evidence rather than intuition.

Finally, we added one custom diagnostic metric beyond the provided evaluator: join coverage. The metric computes precision, recall, and F1 on foreign-key-induced table edges among the predicted tables, and is implemented in `scripts/eval_custom_metrics.py`. On the best confirmed validation run, join coverage was:

- Precision_J: 0.9208
- Recall_J: 0.9686
- F1_J: 0.9125

This metric was useful for diagnosing whether an output missed an entire join path even when some individual tables were correct.

b. Final Pipeline Architecture

The final model family was `Qwen/Qwen2.5-1.5B-Instruct` with a QLoRA adapter trained on the filtered-schema dataset. The best confirmed configuration used:

- Base model: `Qwen/Qwen2.5-1.5B-Instruct`
- Adapter type: QLoRA / PEFT LoRA
- Quantization: 4-bit NF4
- Epochs: 2
- Learning rate: 1.5×10^{-4}
- LoRA rank: 16
- LoRA alpha: 32
- LoRA dropout: 0.05
- Max sequence length: 1024
- Batch size: 1 with gradient accumulation 8

The training entrypoint is `scripts/train_qlora.py`. It supports UTF-8-safe re-exec on Windows, repo-local Hugging Face caches, BF16 capability checks, single-GPU placement, 4-bit loading, and Qwen3 `enable_thinking=False` handling.

At inference time, the pipeline in `main.py` works as follows:

1. Load the input JSON and normalize file decoding with `utf-8-sig`.
2. Load the Spider-format schema for each `db_id` via `schema_utils.py`.
3. Run the deterministic schema filter to select likely tables and columns.
4. Build a compact prompt containing only the filtered schema plus FK relationship lines using `prompting.py`.
5. Generate a JSON candidate with the adapter-backed model, or fall back to a heuristic backend if the ML stack is unavailable.
6. Parse the first JSON object span from the output, repair trivial quote issues, and drop anything schema-invalid using `decoding.py`.

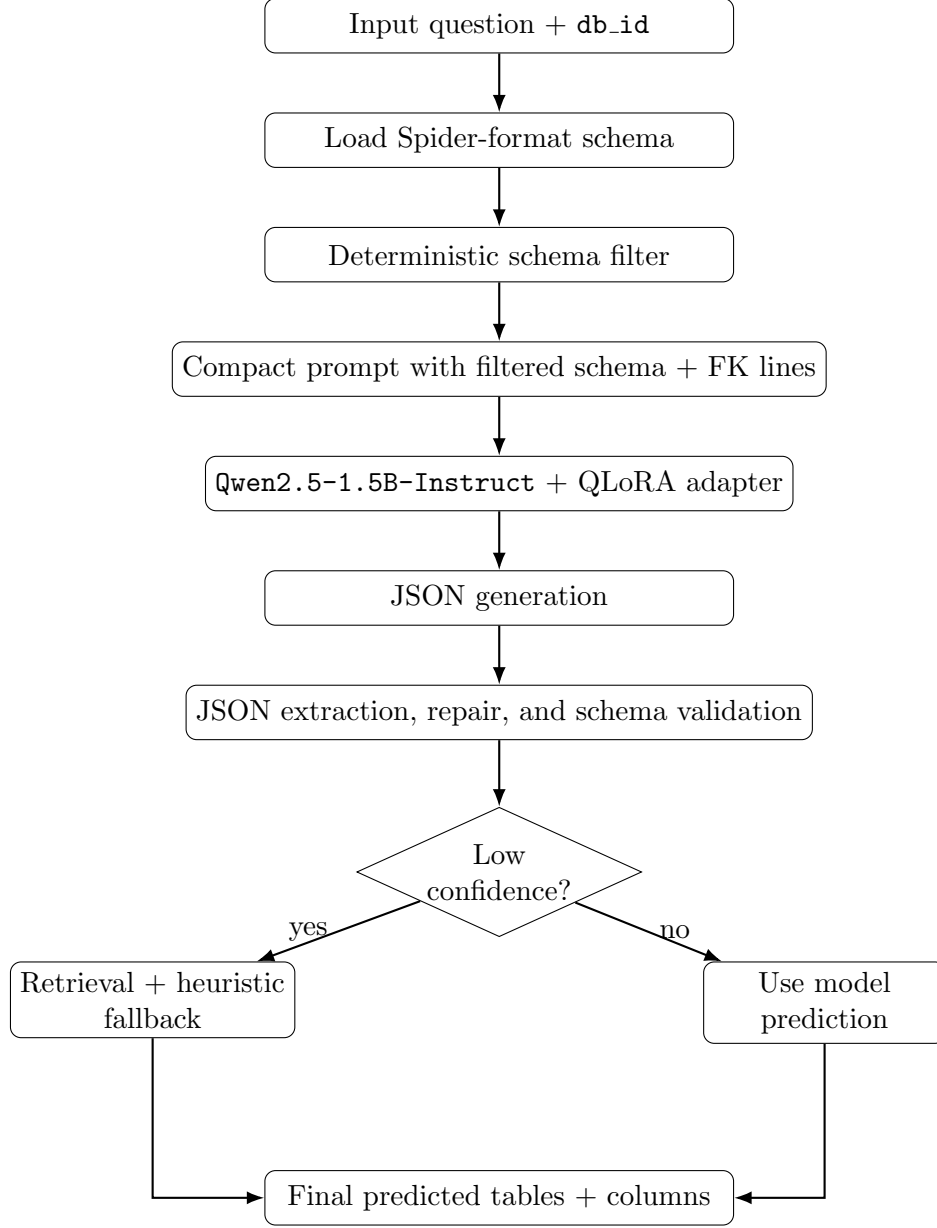


Figure 2: Final inference pipeline. The system first attempts adapter-backed JSON generation using a filtered schema prompt, then applies conservative decoding and validation. Low-confidence generations are replaced with a retrieval-plus-heuristic fallback.

7. Canonicalize table and column names back to schema casing.
8. If generation fails or looks low-confidence, fall back to a retrieval-plus-heuristic prediction.

The fallback path has two pieces:

- A lexical heuristic extractor in `heuristics.py`
- A same-database retriever in `retrieval.py`, which uses token overlap over training questions and IDF-like table weighting

An important late-stage change was a low-confidence gate in `main.py`. The gate swaps to the fallback prediction when the model output is empty, unsupported by retrieval/heuristics, or obviously over-broad. By the end of the project, this gate was not just an offline replay trick: the EC2 rerun through the actual `main.py` entrypoint reproduced the final best score reported in Section d.

Output decoding and post-processing decisions were conservative by design:

- JSON only
- first-object extraction instead of trusting full generations
- canonical schema casing
- hallucinated identifiers dropped
- duplicate columns removed
- non-list values coerced to empty-list table predictions

These details mattered because the handout explicitly warned that scores below about 0.50 often come from pipeline bugs rather than model weakness.

c. Experimentation Methodology

All experiments were run as explicit config sweeps on the same EC2 GPU environment after the training and inference stack was stabilized. I generated a focused config matrix around the strongest family in `scripts/generate_experiment_configs.py` and then launched individual runs with per-config EC2 scripts in `artifacts/`.

The goal of the sweep was not to maximize the number of knobs changed at once. Instead, I moved one axis at a time:

- model scale: 0.5B vs 1.5B vs 1.7B
- schema serialization: filtered vs full
- optimization: 1, 2, or 5 epochs; lower or higher learning rate
- adapter capacity: LoRA rank 16 vs 32
- data strategy: original released train split vs augmented-balanced training set

This one-factor-at-a-time policy was deliberate. After each result, I chose the next experiment to answer a specific question raised by the previous run. For example, once `Qwen2.5-1.5B` clearly beat `0.5B`, later runs stayed in the 1.5B family and tested whether the next gain came from more epochs, lower learning rate, larger LoRA rank, broader schema context, or changed training data.

I did install RapidFire AI on the EC2 environment and kept the trainer compatible with `report_to`, `run_name`, and `logging_dir` hooks, but because of time and infrastructure friction I executed the final sweep as a controlled series of per-config runs rather than a single multi-config API call. The important methodological point is that the runs were systematic, reproducible, and comparable: same hardware, same validation script, same schema assets, same output evaluation.

Model selection also followed a consistent policy. I scored every candidate on the released validation split with the provided evaluator, retained the raw JSON outputs and per-question scored CSVs under `logs/`, and selected the final submission path based on leaderboard score first, then on stability of the TA-facing `main.py` runtime. That matters because this project grades inference, not training logs: a configuration was only considered “final” once the same path worked through the actual endpoint.

Table 1 lists the eight distinct scored configurations used for model selection.

ID	Base model	Schema	Train set	Epochs	LR	LoRA r	Score
C1	Qwen2.5-0.5B	filtered	original	1	2e-4	16	0.4314
C2	Qwen2.5-0.5B	filtered	original	1	1e-4	16	0.3943
C3	Qwen2.5-1.5B	filtered	original	1	2e-4	16	0.5139
C4	Qwen2.5-1.5B	filtered	original	2	1.5e-4	16	0.5279
C5	Qwen2.5-1.5B	filtered	original	5	1.5e-4	16	0.4400
C6	Qwen2.5-1.5B	full	original	5	1.5e-4	16	0.3453
C7	Qwen2.5-1.5B	filtered	augmented-balanced	2	1.5e-4	16	0.4345
C8	Qwen2.5-1.5B	filtered	original	2	1.5e-4	32	0.4880

Table 1: Scored configurations used for model selection.

The corresponding raw JSON outputs retained for these scored configs are:

- C1: `logs/raw_outputs/validation_predictions_ec2_qwen25_0p5b_filtered_e1.json`
- C2: `logs/raw_outputs/validation_predictions_ec2_qwen25_0p5b_filtered_lr1e4.json`
- C3: `logs/raw_outputs/validation_predictions_ec2_qwen25_1p5b_filtered_e1.json`
- C4: `logs/raw_outputs/final_validation_predictions_best_adapter.json`
- C5: `logs/raw_outputs/validation_predictions_ec2_qwen25_1p5b_filtered_e5_lr15e5.json`
- C6: `logs/raw_outputs/validation_predictions_ec2_qwen25_1p5b_full_e5_lr15e5.json`
- C7: `logs/raw_outputs/validation_predictions_ec2_qwen25_1p5b_augbalanced_filtered_e2_lr15e5`
- C8: `logs/raw_outputs/validation_predictions_ec2_qwen25_1p5b_filtered_e2_lr15e5_r32.json`

For the final verified runtime path, the strongest saved output JSON is:

- F1: `logs/raw_outputs/final_validation_predictions_best_adapter_gated.json`

I also trained a Qwen/Qwen3-1.7B filtered run with `enable_thinking=False`, but its training-side metrics were clearly weaker and I did not spend additional budget treating it as a final candidate.

d. Results and Analysis

The most important result is that model scale and schema serialization mattered more than augmentation or bigger adapters.

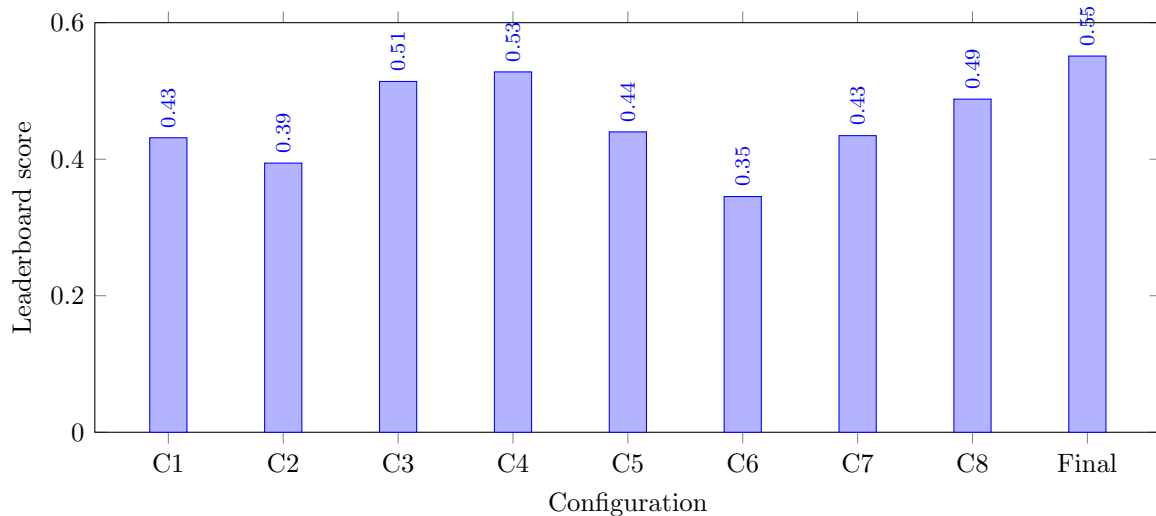


Figure 3: Validation leaderboard scores across model-selection configurations. The best raw checkpoint was C4, while the final confidence-gated runtime path achieved the highest end-to-end score.

The clear win was moving from Qwen2.5-0.5B to Qwen2.5-1.5B. The 0.5B baseline reached only 0.4314, while the first 1.5B filtered run already jumped to 0.5139. This indicates that the task benefits from a stronger instruction model even when the supervision set is small.

The second major result is that filtered-schema prompting was much better than full-schema prompting for this setup. Full-schema training at 5 epochs collapsed to 0.3453, while filtered training remained much stronger. That matches the intuition behind the project: the SAP-style schemas are large enough that reducing prompt clutter is valuable.

The third result is that moderate training helped, but too much training hurt. Moving from 1 epoch to 2 epochs improved 1.5B filtered performance from 0.5139 to 0.5279. However, pushing to 5 epochs reduced the score to 0.4400, strongly suggesting overfitting or degradation in precision.

The fourth result is that naive augmentation and balancing did not help. The augmented-balanced dataset increased the training set to 724 examples, but the resulting model scored only 0.4345. The error pattern was consistent with recall increasing while precision fell sharply, which means the synthetic variants were not clean enough to help this extraction task.

The fifth result is that simply increasing adapter capacity did not help either. Raising LoRA rank from 16 to 32 reduced performance from 0.5279 to 0.4880. In this case the bottleneck was not adapter expressivity.

The best confirmed training configuration remained C4:

- Qwen2.5-1.5B-Instruct
- filtered schema serialization
- original released training set
- 2 epochs
- learning rate 1.5×10^{-4}
- LoRA rank 16

Its confirmed validation breakdown as a raw trained checkpoint was:

- Table score: 0.6122
- Column score: 0.4436
- Leaderboard score: 0.5279

The final submission path added the confidence-gated runtime behavior on top of that best training configuration, and this time the improvement was reproduced through the TA-style entry-point. Running `python main.py --input validation_input.json --output ...` on EC2 with the verified e2 adapter produced:

- Table score: 0.6314
- Column score: 0.4708
- Leaderboard score: 0.5511

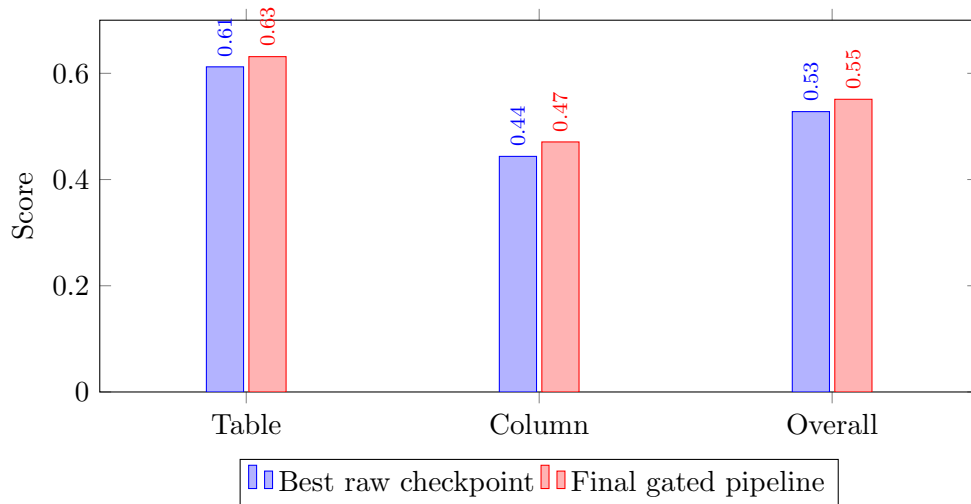


Figure 4: Comparison between the best raw trained checkpoint and the final gated submission pipeline. The gated inference path improved table, column, and overall scores.

This distinction is important for methodology. The report is not claiming a better score from an untracked notebook tweak; it is claiming a better score from the exact inference path that the TAs will run. In other words, the final model choice was not just “best checkpoint wins”; it was “best end-to-end submission pipeline wins.”

In terms of which knobs mattered most:

- Table-level metrics were most sensitive to model scale and schema serialization.
- Column-level metrics were most sensitive to precision-oriented post-processing and overtraining.
- Data augmentation and larger LoRA rank were both negative in this project.

With more time, I would focus on two directions:

1. a reproducible constrained-decoding or confidence-gated inference path that is validated live, not just offline
2. more targeted data augmentation, such as high-precision join-heavy paraphrases or database-specific balancing, rather than broad paraphrase expansion

e. AI Tools Statement

I used AI coding assistants during development to accelerate implementation, debugging, and experiment orchestration. Concretely, I used them to:

- scaffold scripts and refactor utility code
- debug environment and training issues across Windows, Kaggle, and EC2
- summarize experiment outcomes and compare configurations
- draft report language from the actual code and measured results

I did not treat AI outputs as authoritative. All substantial code paths were inspected manually, and all reported metrics in this document come from running the repository’s own training and evaluation scripts.